Communication Topologies in Parallel Dot Product

Federico Teran

March 6, 2026

Introduction

In High-Performance Computing, executing tasks simultaneously is only half the challenge; efficiently sharing data among processes dictates the true scalability of a system. This essay contrasts our sequential dot product implementation with parallel MPI versions, focusing on how different data communication topologies impact performance.

1. Code Level Analysis

At the code level, the sequential version (`sequential.c`) calculates the dot product seamlessly inside a simple `for` loop. In contrast, the MPI versions require the root process to evenly split the input vectors (which are guaranteed to be powers of two) and manually handle array pointers. Our parallel codes integrate the sequential function to validate the result, demonstrating that while parallel code introduces complex point-to-point orchestration (`MPI_Send` / `MPI_Recv`), the underlying mathematical kernel remains identical.

2. Data Communication and Processing Paradigms

The sequential approach relies entirely on a single Processing Element (PE). There is zero data communication overhead, but processing is severely bottlenecked by a single CPU core.

The distributed memory paradigm, implemented via MPI, distributes the workload across a cluster of Computing Nodes (CNs).

Here, memory is private to each process. Processing becomes highly efficient because each PE only computes a fraction of the vectors. However, the paradigm shifts the bottleneck from the CPU to the Interconnection Network (IN). Data must be actively serialised and transmitted over the network, making the communication strategy critical.

3. Paradigmatic Approaches: Naive vs. Binomial Tree

To manage this network overhead, we tested two point-to-point routing algorithms:

- **The Brute-Force Method:** In `bruteforce_parallel.c`, the root process acts as a central hub. If we have p processes, distributing and gathering data takes $p-1$ steps. This linear communication model forces most processes to wait idly while the root sequentially services others, heavily underutilizing the IN's bandwidth.

- **The Binomial Tree Method:** In `tree_parallel.c`, we implemented a hierarchical routing model based on hypercube topologies. Rather than relying solely on the root, nodes act as relays. By doubling the stride length in each step (`step *= 2`), the required communication cycles are reduced from linear to logarithmic time. This allows parallel

message transmission, perfectly balancing the communication workload across the parallel machine.

SProcess-based Parallelisation and MPI Paradigms

Brayan Zuluaga

March 6, 2026

Introduction

Scaling computational performance requires transitioning from simple process-based parallelisation within a single compute node to true distributed memory environments. This essay analyses our implementations of the dot product algorithm, comparing a standard sequential approach against two MPI-based distributed models, evaluating their code structure, computing paradigms, and communication topologies.

1. Analysis of the Code Structure

Our sequential implementation serves as our "golden reference". It is computationally straightforward, calculating the dot product in a single core. However, it scales poorly in execution time as the vector size grows. In our parallel versions (`bruteforce_parallel.c` and `tree_parallel.c`), the code complexity increases because only the root node initially holds the dataset. We had to implement explicit memory chunking and manage local buffers for each process. Our code ensures data correctness by running the sequential function at the end on the root node and comparing it to the parallel aggregated result.

2. Communication and Processing Paradigms

The sequential program operates within a shared memory space, executing as a single process. While one could use system calls like `fork()` to spawn multiple Linux processes within a single node, this scale-up approach does not significantly improve execution time for memory-bound tasks.

To achieve a true scale-out architecture, we utilised the Message Passing Interface (MPI), the standard in the High-Performance Computing (HPC) community. In this paradigm, processes are spawned across multiple Computing Nodes (CNs) in a Non-Uniform Memory Access (NUMA) architecture.

Because these nodes do not share physical memory, Processing Elements (PEs) must communicate explicit messages over an Interconnection Network (IN). The fundamental trade-off in our parallel code is balancing the accelerated processing time against the overhead of moving data across this network.

3. Approaches Within the MPI Paradigm

Since collective operations were restricted, we implemented explicit point-to-point communications using two distinct approaches:

- **Naive (Brute-Force) Approach:** The root process iterates over all $p - 1$ processes to send their chunks and later receives the partial dot products sequentially. This linear communication complexity creates a massive bottleneck at the root node.

- **Binomial Tree (Incomplete Hypercube) Approach:** To speed up communications, we structured the processes logically as an incomplete hypercube.

  Instead of the root communicating with everyone, processes exchange data in $\log_2(p)$ steps. Multiple messages are sent in parallel across the IN during each step, drastically reducing the communication overhead and fully exploiting the parallel machine's capabilities.

Workload Balancing and Distributed Execution

Jhon Mejias

March 6, 2026

Introduction

Distributing a computational problem like a dot product across a multicomputer system introduces significant architectural decisions. This essay provides a comparative analysis of a single-process sequential execution versus message-passing parallelisation using MPI, examining code complexity, paradigm shifts, and communication efficiency.

1. Examining the Implementation Code

The sequential code is deterministic and trivial to debug since all data resides in a contiguous block of memory. However, to convert this into a parallel program, we had to implement a master-worker architecture. Our code strictly ensures that the root process reads the standard input, calculates the chunk sizes (dim/p), and manually dispatches the vectors. A vital aspect of our implementation is the inclusion of the sequential function as a "golden reference" inside the parallel programs to automatically check for data consistency after the distributed operations conclude.

2. Processing Paradigms: Sequential vs. MPI

Executing the algorithm sequentially or via single-node process duplication (e.g., `fork`) fails to provide true scalability. The distributed MPI paradigm solves this by treating the environment as a set of autonomous Computing Nodes (CNs) connected by an Interconnection Network (IN).

In this NUMA (Non-Uniform Memory Access) environment, processing is vastly accelerated because the vector is evenly split amongst multiple Processing Elements (PEs). The trade-off is the introduction of data communication latency, forcing us to carefully engineer how messages are passed to avoid stalling the cores.

3. Communication Approaches

Because collective operations were disallowed, we explored two distinct point-to-point communication methods:

- **Naive Approach:** The root node loops sequentially to send and receive data from every other node. While easy to write, this approach violates the principle of workload balancing. The root node does 100% of the network transmission work, leading to massive congestion and idle time for the receiving CNs.
- **Incomplete Hypercube (Binary Tree):** To truly speed up communications, we distributed the network workload itself.

  By splitting the data hierarchically, nodes that receive data early become senders in the subsequent steps. This cooperative networking scheme balances the load across all PEs,

resolving the root-node bottleneck and transforming a $O(N)$ communication delay into a highly efficient $O(\log N)$ operation.